

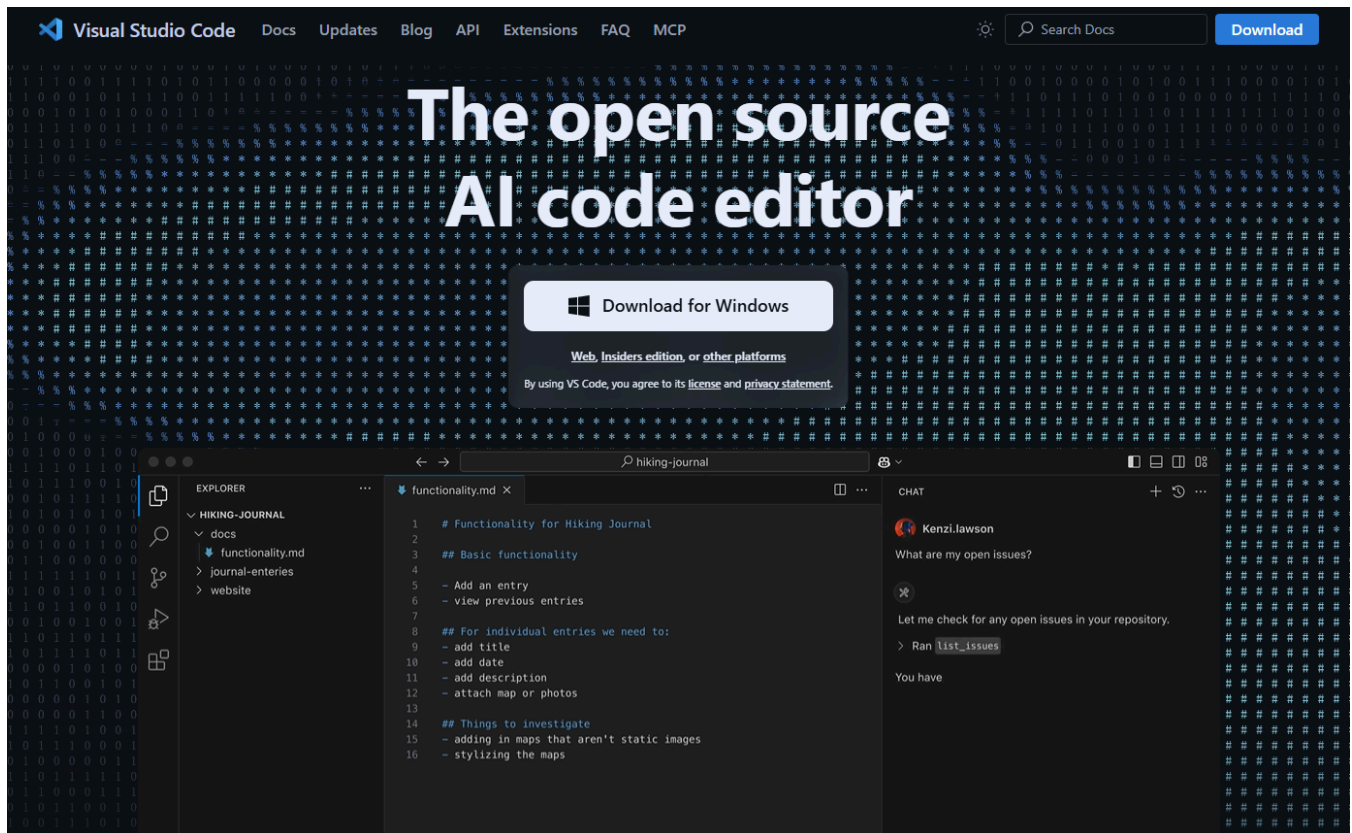
ROS2 Integrated Development Environment Setup

If you are using the factory image we provided, manual installation and configuration are not required.

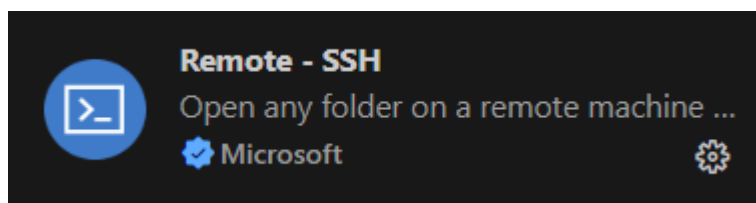
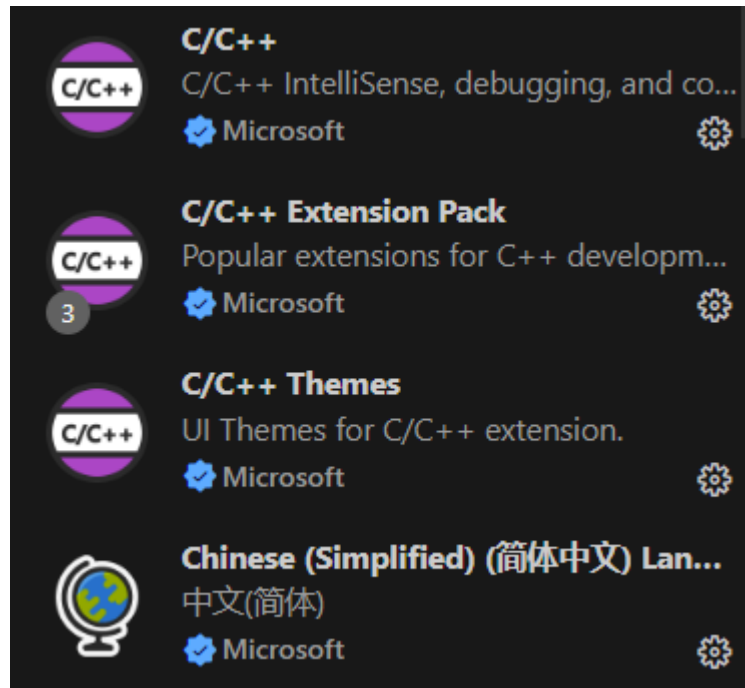
In theory, you can write basic ROS2 programs with just a text editor in ROS2. However, to improve development efficiency, you can install the integrated development environment VSCode.

1. Using VSCode Development Environment

- Search for VSCode in your browser, download the installation package for your platform, and install it.

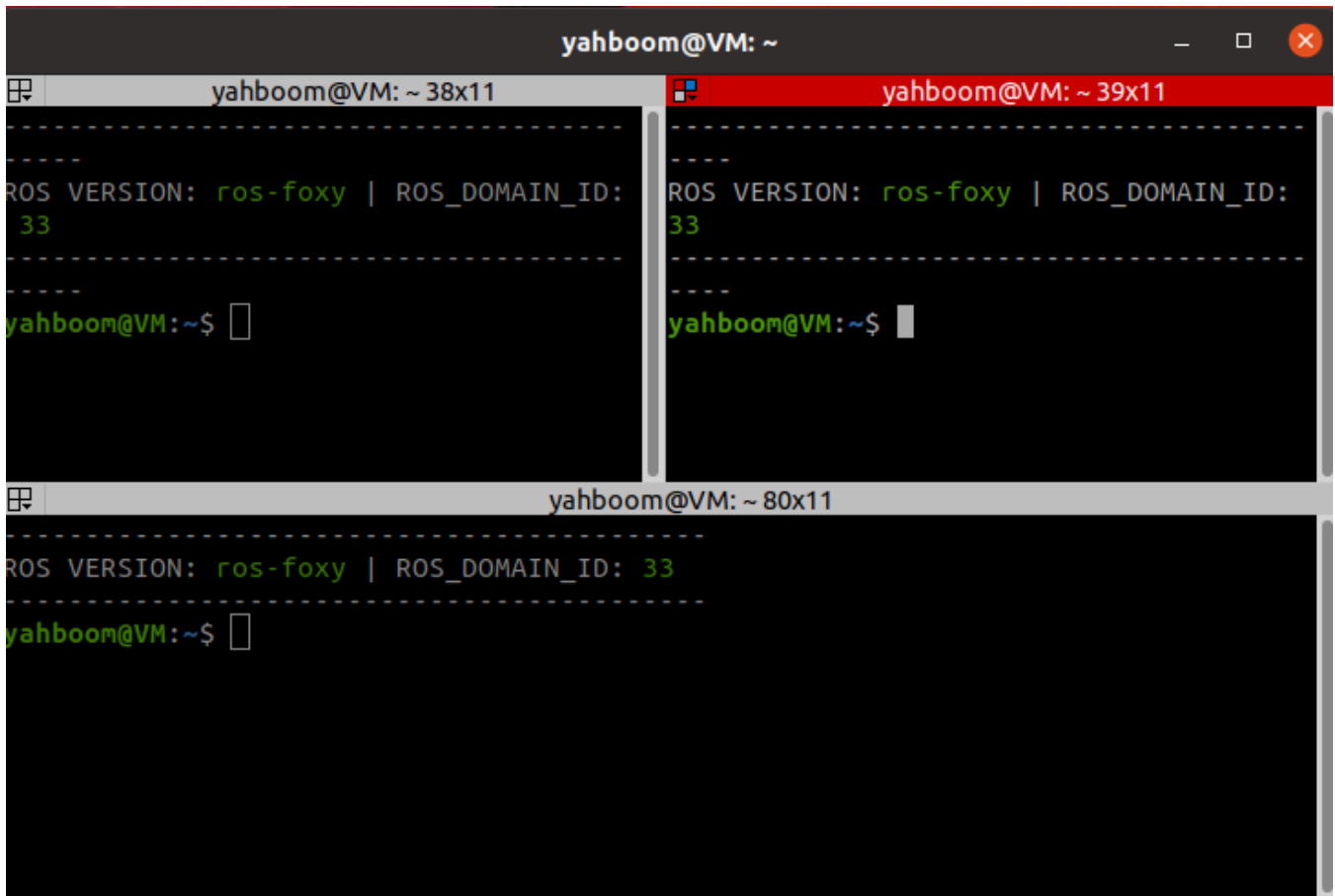


- Install commonly used plugins to help improve work efficiency.



2. Using the Terminal

In ROS2, you need to use the terminal frequently. We recommend a relatively good terminal: Terminator. The effect is as follows:



2.1 Installation

```
sudo apt install terminator
```

2.2 Launch

Press `Ctrl+Alt+T` to launch

2.3 Common Terminator Shortcuts

Operations within the same tab:

Alt+Up	//Move to the terminal above
Alt+Down	//Move to the terminal below
Alt+Left	//Move to the terminal on the left
Alt+Right	//Move to the terminal on the right
Ctrl+Shift+O	//Split terminal horizontally
Ctrl+Shift+E	//Split terminal vertically
Ctrl+Shift+Right	//Move the split bar to the right in vertically split
terminals	
Ctrl+Shift+Left	//Move the split bar to the left in vertically split
terminals	
Ctrl+Shift+Up	//Move the split bar up in horizontally split terminals
Ctrl+Shift+Down	//Move the split bar down in horizontally split terminals
Ctrl+Shift+S	//Hide/show scrollbar
Ctrl+Shift+F	//Search

```

Ctrl+Shift+C          //Copy selected content to clipboard
Ctrl+Shift+V          //Paste clipboard content here
Ctrl+Shift+W          //Close current terminal
Ctrl+Shift+Q          //Exit current window, all terminals in the current window
will be closed
Ctrl+Shift+X          //Maximize current terminal
Ctrl+Shift+Z          //Maximize current terminal and enlarge font
Ctrl+Shift+N or Ctrl+Tab //Move to next terminal
Ctrl+Shift+P or Ctrl+Shift+Tab //Move to previous terminal

```

Operations between tabs:

```

F11                  //Toggle fullscreen
Ctrl+Shift+T         //Open a new tab
Ctrl+PageDown        //Move to next tab
Ctrl+PageUp          //Move to previous tab
Ctrl+Shift+PageDown  //Swap current tab with the next tab
Ctrl+Shift+PageUp    //Swap current tab with the previous tab
Ctrl+Plus (+)        //Increase font size
Ctrl+Minus (-)       //Decrease font size
Ctrl+Zero (0)        //Restore font to original size
Ctrl+Shift+R         //Reset terminal state
Ctrl+Shift+G         //Reset terminal state and clear screen
Super+g              //Bind all terminals so that input goes to all terminals
Super+Shift+G        //Unbind
Super+t              //Bind all terminals in the current tab, content entered in
one terminal will automatically enter other terminals
Super+Shift+T        //Unbind
Ctrl+Shift+I         //Open a new window, the new window uses the same process as
the original window
Super+i              //Open a new window, the new window uses a different process
from the original window

```

3. Using Git

3.1 Installation

In daily work, because it is team collaboration and involves version control, git is an indispensable skill. Git is a free and open-source distributed version control system. To install git on Ubuntu:

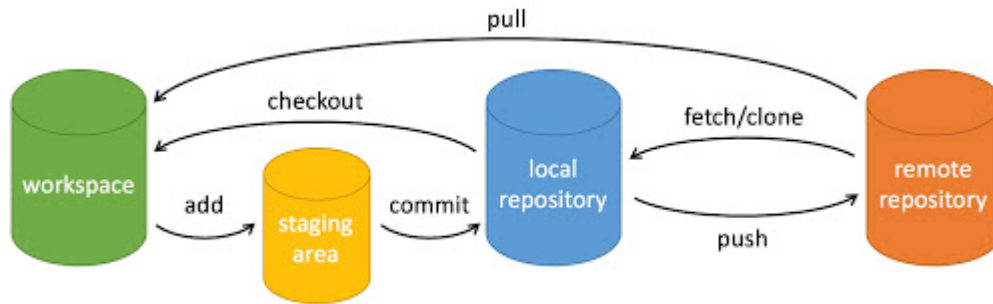
```
sudo apt install git
```

3.2 Git Basic Operations

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

This chapter will introduce commands for creating and committing snapshots of your project.

The six most commonly used Git commands are: **git clone**, **git push**, **git add**, **git commit**, **git checkout**, **git pull**. We will introduce them in detail later.



Explanation:

- workspace: Working area
- staging area: Staging area/buffer area
- local repository: Version library or local repository
- remote repository: Remote repository

A simple operation steps:

```
$ git init
$ git add .
$ git commit
```

- git init - Initialize repository.
- git add . - Add files to staging area.
- git commit - Add staging area content to repository.

3.2.1 Create Repository Commands

The following table lists Git commands for creating repositories:

Command	Description
<code>git init</code>	Initialize repository
<code>git clone</code>	Copy a remote repository, i.e., download a project.

3.2.2 Commit and Modify

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

The following table lists commands for creating and committing snapshots of your project:

Command	Description
<code>git add</code>	Add files to staging area
<code>git status</code>	View current status of repository, showing changed files.
<code>git diff</code>	Compare differences between files, i.e., differences between staging area and working area.

Command	Description
<code>git commit</code>	Commit staging area to local repository.
<code>git reset</code>	Revert to previous version.
<code>git rm</code>	Delete files from staging area and working area.
<code>git mv</code>	Move or rename files in working area.
<code>git checkout</code>	Branch switching.
<code>git switch</code> (Git 2.23 and later)	Switch branches more clearly.
<code>git restore</code> (Git 2.23 and later)	Restore or undo file changes.

3.2.3 Commit Log

Command	Description
<code>git log</code>	View historical commit records
<code>git blame <file></code>	View historical modification records of specified file in list format

3.2.4 Remote Operations

Command	Description
<code>git remote</code>	Remote repository operations
<code>git fetch</code>	Fetch code from remote
<code>git pull</code>	Download remote code and merge
<code>git push</code>	Upload remote code and merge

For more information on using the git tool, enter `git --help` in the terminal to view the help documentation.